Table 5: Test accuracies (%) for GNNs with 2, 4, 8, 16, and 32 layers. For each layer, the best accuracy is marked in bold, and the second best is underlined. Results are averaged over 5 runs. We used the default hyperparameters of the ContraNorm method [14] without additional optimizations

Dataset	Model	#L=2	#L=4	#L=8	#L=16	#L=32
Cora	Vanilla GCN	**81.75 ± 0.51**	72.61 ± 2.42	17.71 ± 6.89	20.71 ± 8.54	19.69 ± 9.54
	w/ LayerNorm [2]	79.96 ± 0.73	**77.45 ± 0.67**	39.09 ± 4.68	7.79 ± 0.00	7.79 ± 0.00
	w/ PairNorm [62]	75.32 ± 1.05	72.64 ± 2.67	71.86 ± 3.31	54.11 ± 9.49	36.62 ± 2.73
	w/ ContraNorm [14]	79.75 ± 0.33	77.02 ± 0.96	74.01 ± 0.64	68.75 ± 2.10	46.39 ± 2.46
	w/ Ours	79.25 ± 0.03	77.35 ± 0.05	**74.85 ± 0.02**	**71.15 ± 0.02**	**52.12 ± 0.07**
CiteSeer	Vanilla GCN	**69.18 ± 0.34**	55.01 ± 4.36	19.65 ± 0.00	19.65 ± 0.00	19.65 ± 0.00
	w/ LayerNorm	63.27 ± 1.15	60.91 ± 0.76	33.74 ± 6.15	19.65 ± 0.00	19.65 ± 0.00
	w/ PairNorm	61.59 ± 1.35	53.01 ± 2.19	55.76 ± 4.45	44.21 ± 1.73	36.68 ± 2.55
	w/ ContraNorm	64.06 ± 0.85	60.55 ± 0.72	59.30± 0.67	49.01 ± 3.49	36.94 ± 1.70
	w/ Ours	65.45 ± 0.08	**61.75 ± 0.02**	**60.08 ± 0.01**	**52.09 ± 0.05**	**40.44 ± 0.08**
Chameleon	Vanilla GCN	45.79 ± 1.20	37.85 ± 1.35	22.37 ± 0.00	22.37 ± 0.00	23.37 ± 0.00
	w/ LayerNorm	63.95 ± 1.29	55.79 ± 1.25	34.08 ± 2.62	22.37 ± 0.00	22.37 ± 0.00
	w/ PairNorm	62.24 ± 1.73	58.38 ± 1.48	49.12 ± 2.32	37.54 ± 1.70	30.66 ± 1.58
	w/ ContraNorm	64.78 ± 1.68	58.73 ± 1.12	48.99 ± 1.52	40.92 ± 1.74	35.44 ± 3.16
	w/ Ours	**66.06 ± 0.04**	**60.12 ± 0.03**	**51.20 ± 0.05**	**46.15 ± 0.03**	**39.05 ± 0.11**
Squirrel	Vanilla GCN	29.47 ± 0.96	19.31 ± 0.00	19.31 ± 0.00	19.31 ± 0.00	19.31 ± 0.00
	w/ LayerNorm	43.04 ± 1.25	29.64 ± 5.50	19.63 ± 0.45	19.96 ± 0.44	19.40 ± 0.19
	w/ PairNorm	43.86 ± 0.41	40.25 ± 0.55	36.03 ± 1.43	29.55 ± 2.19	29.05 ± 0.91
	w/ ContraNorm	47.24 ± 0.66	40.31 ± 0.74	35.85 ± 1.58	32.37 ± 0.93	27.80 ± 0.72
	w/ Ours	**48.32 ± 0.03**	**43.72 ± 0.07**	**38.12 ± 0.08**	**35.02 ± 0.03**	**33.10 ± 0.03**

Table 6: Test accuracies (%) for GNNs with 2, 4, 8, 16, and 32 layers. For each layer, the best accuracy is marked in bold, and the second best is underlined. Results are averaged over 5 runs. We optimized the hyperparameters of each one of the methods independently.

Dataset	Model	#L=2	#L=4	#L=8	#L=16	#L=32
Cora	Vanilla GCN	**81.30 ± 0.001**	81.22 ± 0.022	78.50 ± 0.020	72.80 ± 0.010	20.05 ± 0.065
	w/ LayerNorm [2]	80.12 ± 0.002	79.22 ± 0.002	75.20 ± 0.011	67.95 ± 0.039	15.20 ± 0.092
	w/ PairNorm [62]	80.65 ± 0.002	78.23 ± 0.065	74.80 ± 0.001	72.10 ± 0.033	61.12 ± 0.032
	w/ ContraNorm [14]	82.23 ± 0.030	79.75 ± 0.004	76.45 ± 0.004	74.35 ± 0.018	65.44 ± 0.027
	w/ Ours	82.44 ± 0.002	**82.02 ± 0.002**	**79.98 ± 0.054**	**75.01 ± 0.017**	**71.33 ± 0.027**
CiteSeer	Vanilla GCN	69.12 ± 0.001	61.65 ± 0.004	58.35 ± 0.011	30.02 ± 0.030	17.44 ± 0.011
	w/ LayerNorm	66.86 ± 0.029	65.12 ± 0.001	60.02 ± 0.015	20.21 ± 0.022	17.55 ± 0.013
	w/ PairNorm	67.55 ± 0.012	65.32 ± 0.012	60.98 ± 0.005	54.88 ± 0.002	40.65 ± 0.037
	w/ ContraNorm	**69.45 ± 0.002**	64.65 ± 0.035	58.98 ± 0.025	54.87 ± 0.017	48.95 ± 0.002
	w/ Ours	69.20 ± 0.009	**67.87 ± 0.001**	**64.65 ± 0.001**	**60.94 ± 0.054**	**57.12 ± 0.021**
Chameleon	Vanilla GCN	66.23 ± 0.035	63.82 ± 0.015	47.59 ± 0.050	35.09 ± 0.002	22.37 ± 0.001
	w/ LayerNorm	66.03 ± 0.015	61.18 ± 0.020	49.78 ± 0.034	29.82 ± 0.007	22.37 ± 0.001
	w/ PairNorm	67.32 ± 0.077	63.83 ± 0.015	54.17 ± 0.012	46.49 ± 0.023	41.01 ± 0.015
	w/ ContraNorm	66.50 ± 0.003	60.93 ± 0.098	54.17 ± 0.072	51.54 ± 0.055	41.45 ± 0.089
	w/ Ours	**66.98 ± 0.001**	**65.76 ± 0.015**	**59.98 ± 0.035**	**54.65 ± 0.005**	**50.11 ± 0.013**

4 Experiments

We provide a series of experiments to validate the advantages of using a centered self-attention layer compared to standard self-attention layers, including simulations, WSSS, and GNNs.

4.1 Simulations

In order to gain a better understanding of the impact of the importance of the offset parameter on oversmoothing, we conducted a series of experiments, using randomly initialized Transformers. These experiments study the multiple pre-existing variations of Transformer, as described in [54] (Pre-LN, Post-LN and ResiDual).

We denote by $R_1 = X_1 \in \mathbb{R}^{n\times d}$ the input to the network, which includes n tokens of dimension of d, and by $S_\gamma(X;W) = \text{softmax}\left(\frac{XW_Q(XW_K)^\top}{\sqrt{d}} + \gamma\frac{\mathbb{1}\mathbb{1}^\top}{n}\right)XW_V$ a self-attention layer applied to X with offset $\gamma \in \mathbb{R}$. Three types of Transformer architectures, f, are considered: (i) The Pre-LN

architecture $f(X_1) = N(X_{L+1})$, where each layer computes the function $X_{i+1} = S_\gamma(N(X_i); W^i) + X_i$. (ii) The Post-LN architecture takes the form $f(X_1) = X_{L+1}$, where $X_{i+1} = N(S_\gamma(X_i; W^i) + X_i)$. (iii) The ResiDual architecture $f(X_1) = N(R_{L+1}) + X_{L+1}$, where $X_{i+1} = N(S_\gamma(X_i; W^i) + X_i)$ and $R_{i+1} = R_i + S_\gamma(X_i; W^i)$. Here, $N : \mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d}$ is a function that normalizes each row individually by dividing it by its norm, and $\gamma \in \mathbb{R}$ is an offset parameter.

For our experiments, we used the identity matrix $X_1 \in \mathbb{R}^{100\times 100}$ as the input to the network, representing $n = 100$ tokens, each with a dimension of $d = 100$. We employed rectangular weight matrices $W^i_Q, W^i_K, W^i_V \in \mathbb{R}^{100\times 100}$. Our focus was on investigating the rank of the matrix X_i generated by the network after the ith layer. To measure the rank of a given matrix A, we counted the number of singular values of the matrix $\frac{A}{\|A\|_F}$ (where $\|\cdot\|_F$ denotes the Frobenius norm) that exceeded the threshold $\epsilon = 10^{-3}$.

In the first simulation, we used identity weight matrices $W^i_Q = W^i_K = W^i_V = I$, while in the second simulation, we sampled W^i_Q, W^i_K, and W^i_V from a uniform distribution $U([0, 1])^{100\times 100}$. Figure 1 illustrates the rank of the output of the network as a function of the coefficient γ (ranging from -1.5 to 1.5 on the x-axis) and the depth (ranging from 1 to 2000 on the y-axis). Evidently, in both cases, for any coefficient $\gamma > -1$, the rank of the output matrix decreases to 1 as we progress deeper into the network. This is avoided for any $\gamma \leq -1$. Therefore, we conclude that each one of the different variations of the Transformer architecture suffers from oversmoothing that is largely resolved when our correction is used.

4.2 WSSS experiments

The WSSS pipeline presented in Sec. 3.3 is less complex than recent methods [41]. It is evaluated on the PASCAL VOC 2012 dataset [11], which is a widely used benchmark in semantic segmentation. Following common practice, the VOC 2012 dataset is further augmented with the SBD dataset [16]. The augmented dataset consists of 10,582 training images, 1,449 validation images, and 1,456 test images. Training uses only the image-level labels. Evaluation is performed via the standard mIoU.

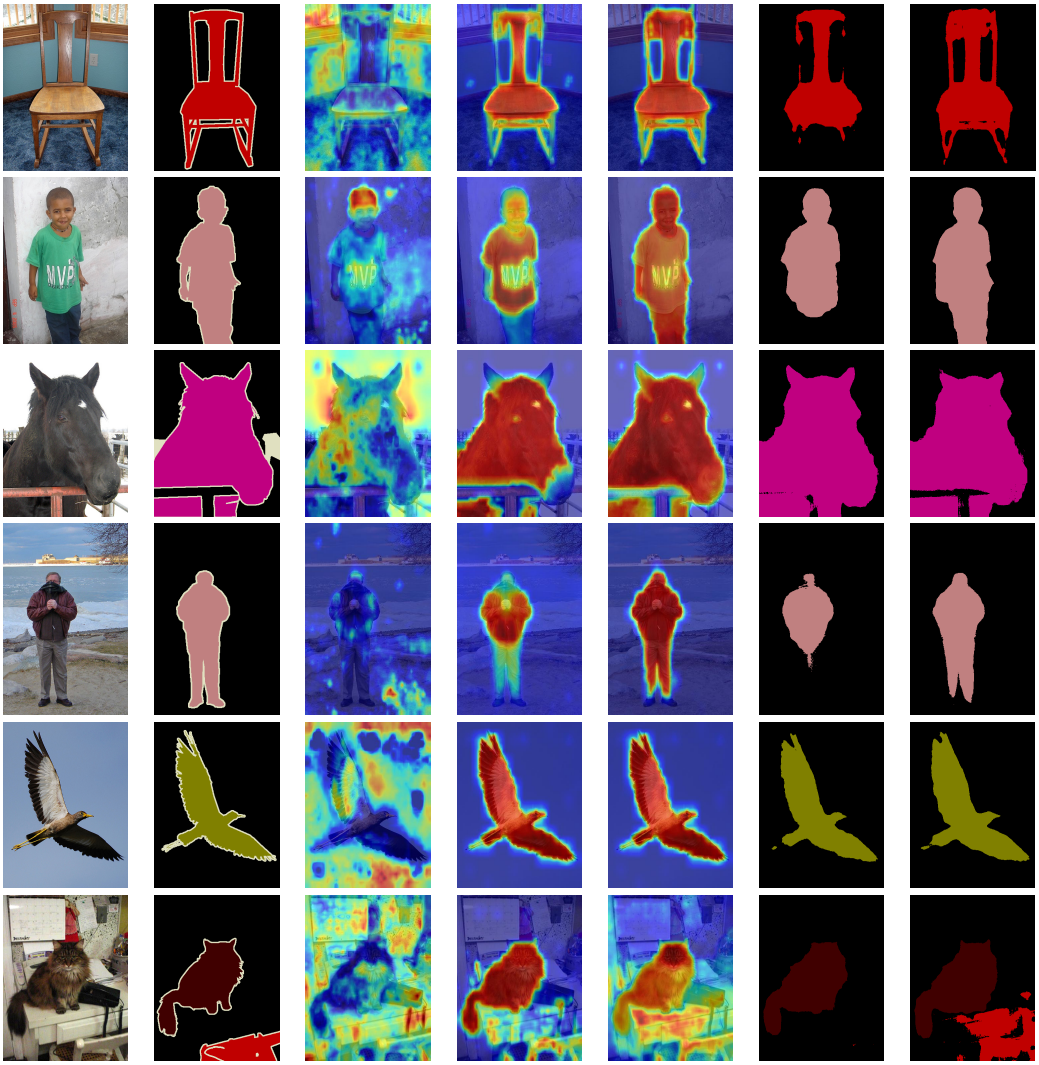
Evaluation is performed using a DeiT-small network [48] pretrained on ImageNet-1K. The decoder in the proposed architecture is a simple segmentation head, which comprises two 3×3 convolutional layers with a dilation rate of 5, followed by a 1×1 prediction layer. We set the auxiliary classifier to operate on top of the third to the last layer of the neural network. Training is performed with the AdamW optimizer [31], with a learning rate that increases linearly to $6e^{-5}$ during the first 2500 iterations, and subsequently decays with a polynomial scheduler. The warm-up rate is set to $1e^{-6}$, and the decay rate is set to 0.9. A batch size of 4 is used for a total of 80k iterations.

The results are presented in Tab. 1. Our method achieves a mIoU of 72.5 and 72.9 on the validation and test sets, respectively, compared to 71.1 and 72.2 achieved by the recent state-of-the-art single-stage method [41]. Additionally, we observe that our approach outperforms most of the multi-stage methods, with the exception of a method that employs additional supervision in the form of saliency maps (I + S). Note that ToCo, [41] also report results with Deit-S; however, these results were lower than what is reported in the table. Tab. 2 lists the semantic segmentation results on the validation set for each class in the dataset. Tab. 3 presents the results at the level of the pseudo labels produced on the training set, demonstrating the effectiveness of the solution earlier on in the pipeline.

Fig. 2 presents typical samples of the Class Activation Maps (CAMs) and segmentation masks generated by our proposed method compared to those produced by ToCo [41]. This leading baseline uses complex pipelines to address the oversmoothing issue. Evidently, our method produces semantic segmentations with more precise object boundaries compared to this recent method.

Ablation Studies. We conduct various experiments to examine the contribution of the different parts of our pipeline and to explore the effect of the centering loss for multiple configurations. These results are presented in Tab. 4. First, ViT with a single classification head at the last layer results in CAMs that provide a poor mIoU score of only 28.1 due to oversmoothing. Our correction term alone leads to a significant improvement in mIoU scores, to 56.1.

The auxiliary classification head of the intermediate layer does not help the CAM of the final layer when the centering term is not applied and helps slightly when it is added. We note that the CAM of the intermediate layer itself is much higher than the last layer counterpart, in the case of only $\mathcal{L}_{cls} + \mathcal{L}^m_{cls}$. Here, too, our correction term helps by a significant margin.



(a) (b) (c) (d) **(e)** (f) (g)

Figure 2: Comparison of extracted CAMs and segmentation maps for our method and for ToCo [41]. (a) input, (b) ground-truth mask, (c) CAM of ViT, (d) CAM of ToCo [41], (e) CAM of our method, (f) the masks obtained by ToCo, (g) the masks obtained by our method.

It is evident that both $\mathcal{L}_{seg}$ and $\mathcal{L}_{reg}$ lead to an improvement in performance for the CAM of the last layer. The latter also improves the segmentation results (there is no segmentation head without $\mathcal{L}_{seg}$). Both the last layer CAM and the segmentation head benefit also from the centering term by a significant margin, with or without the regularization term. The CAM of the middle layer, however, shows a paradoxical behavior. While it benefits from the addition of the segmentation loss, both centering and the regularization term hurt its performance. It is likely that by adding the regularization term, the loss term associated with the intermediate head becomes much less prominent since the emphasis becomes the last layer (three loss terms are associated with the last layer). Similarly, the effect of $\mathcal{L}_{cls}^{m}$ among three or four loss term diminishes when adding the centering term, since this term allows for an effective optimization of the loss terms of the last layer (see the $\mathcal{L}_{cls}$ ablation).

4.3 Experiments with GNNs

Following previous work [14], evaluation is performed on two widely used citation graphs, Cora [32] and CiteSeer [12], as well as two Wikipedia article networks, Chameleon and Squirrel [38]. We